



Agentic LLM Code Generation for Industrial Automation: How Much Does Tool Feedback Improve IEC 61131-3 Structured Text?

Industry Collaboration with Everllence SE (formerly MAN Energy Solutions)

Student: Necati Furkan Colak
Program: MSc Computer Science
Supervisor: [Supervisor Name]
Email: necatifurkan.colak@uzh.ch

Student ID: 24746323
Semester: Spring 2026
Co-Supervisor: [Everllence Contact]
Date: July 4, 2026

Abstract

PLC languages (IEC 61131-3 Structured Text [8]) are low-resource and safety-relevant; single-shot LLM generation is demonstrably unreliable for them [1, 2]. The emerging remedy is *agentic* generation: the model iterates against ground-truth tools — compiler and executable tests — repairing its output from their feedback [5, 6]. Published pipelines (LLM4PLC [2], Agents4PLC [3]) show the pattern works but report whole-pipeline results, leaving the practitioner question open: *how much does each feedback loop buy, for which task types, at what cost?* This thesis answers it with a bounded, controlled study: a benchmark of 40–50 Structured Text tasks (from the OSCAT library [9] and genericised Everllence specifications) with hidden test suites on an open toolchain [10], over which four regimes (single-shot, best-of- n , compiler feedback, compiler+test feedback) are compared for one Azure-hosted and one open-weight model. The controlled, cost-annotated ablation — and its transfer to the PLC domain — is the contribution.

Motivation and Gap

Everllence automation engineers translate specifications, cause-and-effect matrices, and signal lists into PLC programs validated through acceptance tests [11]; much of this code is structurally repetitive, yet defects found at commissioning are expensive. The company’s Copilot/Azure licences make LLM assistance available — the open question is *when it can be relied on*. Prior work establishes that ChatGPT produces plausible but inconsistent control logic [1] and that verification-wrapped pipelines improve outcomes [2, 3]; a 21-task benchmark compared universal LLMs on PLC code [4]. But component contributions remain confounded: retry vs. compiler messages vs. failing-test feedback; iteration-count returns; open-weight vs. closed models; task types that stay unreliable regardless. A controlled factorial study on a modest benchmark answers precisely these questions. Formal verification [7] is deferred to an optional extension.

Feasibility

- The study shape — benchmark construction, controlled comparison, statistical analysis — follows established empirical evaluation practice in the LLM-for-code literature [1, 4].
- Benchmark scale is proven: the influential control-logic study used compact prompt collections [1]; the universal LLM comparison used 21 tasks [4]. The proposed 40–50 tasks sit in this range.
- Open toolchain (MATIEC/OpenPLC [10]) removes vendor-licence risk; self-repair loops are documented patterns [5].
- No confidential data: tasks authored from public libraries [9] and genericised specifications; company involvement (realism review, quality rating) is additive.

Research Questions

1. **RQ1:** How much does each feedback level (single-shot, best-of- n , compiler, compiler+tests) improve hidden-test pass rates, for a closed and an open-weight model under identical budgets?
2. **RQ2:** How do improvement and cost scale with iteration count — yielding actionable cost–reliability curves?
3. **RQ3:** Which task types (combinational logic, timers/edges, stateful sequences) remain unreliable with full feedback, and what does failure analysis plus expert code review reveal?

Methodology

- **Benchmark:** 40–50 tasks in three tiers — (a) function-level from OSCAT, (b) component-level (interlocks, sequences, alarm logic) from genericised Everllence specifications, (c) small integration tier. Each: NL spec, FB signature, hidden scan-cycle test suite, reference solution; realism reviewed in one engineer workshop.
- **Regimes:** 4×2 design (four feedback regimes $\times$ closed/open-weight model), fixed max iterations, matched token budgets, identical prompts. No multi-agent roles, no fine-tuning.
- **Evaluation:** pass@1 per regime/model/tier with McNemar tests; tokens/wall-clock per solved task (cost–reliability curves); failure-mode taxonomy from transcripts; blinded expert rating of solution readability by 2–3 engineers.



- **Safety:** all execution in simulation; no generated code approaches physical equipment.
- **Scope/risk:** graphical languages, vendor IDEs, and deployment out of scope. Toolchain risk handled by a Week-1–2 spike (three tasks compile+test end-to-end before benchmark authoring); tier (a) alone already supports RQ1/RQ2 if authoring runs long; local open-weight model prevents API-limit blockage.

Contribution and Value for Everllence

(1) An open benchmark of specification+test Structured Text tasks on a fully open toolchain; (2) a controlled, cost-annotated ablation of agentic feedback levels, converting whole-pipeline claims [2, 3] into component-level evidence; (3) a failure-mode taxonomy and expert-rated quality assessment. For Everllence: an adoption map for its Copilot/Azure investment — per task type, measured reliability, cost per solved task, and the review effort still required.

Timeline

Month	Work
1	Literature review; toolchain spike (3 tasks end-to-end); task typology and guidelines.
2	Benchmark authoring (tiers a–c); reference solutions; realism workshop; freeze.
3	Agent harness (generate/compile/test tools, logging); single-shot and best-of- n baselines.
4	Feedback regimes; full 4×2 run; iteration-budget sweeps.
5	Statistics (RQ1–RQ2); failure taxonomy; expert quality rating (RQ3); optional PLCverif extension.
6	Thesis writing, revision, benchmark/harness release, submission.

References

- [1] H. Kozirolek, S. Gruener, V. Ashiwal. *ChatGPT for PLC/DCS Control Logic Generation*. IEEE ETFA, 2023.
- [2] M. Fakhri et al. *LLM4PLC: Harnessing Large Language Models for Verifiable Programming of PLCs*. ICSE-SEIP, 2024.
- [3] Z. Liu et al. *Agents4PLC: Automating Closed-Loop PLC Code Generation and Verification Using LLM-Based Agents*. arXiv:2410.14209, 2024.
- [4] *Generating PLC Code with Universal Large Language Models*. IEEE, 2024.
- [5] X. Chen, M. Lin, N. Schärli, D. Zhou. *Teaching Large Language Models to Self-Debug*. ICLR, 2024.
- [6] N. Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS, 2023.
- [7] D. Darvas, E. Blanco Viñuela, B. Fernández Adiego. *PLCverif: A Tool to Verify PLC Programs Based on Model Checking*. ICALEPCS, 2015.
- [8] IEC. *IEC 61131-3: Programmable Controllers — Programming Languages*. Geneva.
- [9] OSCAT. *Open Source IEC 61131-3 Function Block Library*. <http://www.oscat.de/>
- [10] E. Tisserant, L. Bessard, M. de Sousa. *An Open Source IEC 61131-3 IDE (Beremiz/MATIEC)*. IEEE INDIN, 2007.
- [11] Everllence SE. *Company and Product Information*. 2025. <https://www.everllence.com/>

Student Signature:

Supervisor Signature: